Partie 1 - Code de l'interface (app.py)

```
import threading
import tkinter as tk
from tkinter import ttk, messagebox

import ciphers
import integrity
import passwords


def parse_hill_key(raw):
    raw = raw.strip()
    if "," in raw or ";" in raw:
        rows = [r for r in raw.replace(";", "\n").split("\n") if r.strip()]
        if len(rows) == 1:
            nums = [int(x) for x in rows[0].replace(";", ",").split(",")]
            size = int(len(nums) ** 0.5)
            if size * size != len(nums):
                raise ValueError("la matrice doit être carrée")
            return [nums[i * size:(i + 1) * size] for i in range(size)]
        return [[int(x) for x in r.split(",")] for r in rows]
    return ciphers.key_from_text(raw)


class MessagingTab(ttk.Frame):
    def __init__(self, parent):
        super().__init__(parent, padding=10)
        top = ttk.Frame(self)
        top.pack(fill="x")
        ttk.Label(top, text="Algorithme").grid(row=0, column=0, sticky="w")
        self.algo = ttk.Combobox(top, values=["Caesar", "Hill", "Playfair"], state="readonly",
            width=12)
        self.algo.current(0)
        self.algo.grid(row=0, column=1, padx=6)
        self.algo.bind("<<ComboboxSelected>>", self.refresh_hint)
        ttk.Label(top, text="Clé").grid(row=0, column=2, padx=(12, 0))
        self.key = ttk.Entry(top, width=24)
        self.key.insert(0, "3")
        self.key.grid(row=0, column=3, padx=6)
        ttk.Label(top, text="Sens").grid(row=0, column=4, padx=(12, 0))
        self.direction = ttk.Combobox(top, values=["droite", "gauche"], state="readonly", width=7)
        self.direction.current(0)
        self.direction.grid(row=0, column=5, padx=6)
        self.hint = ttk.Label(self, text="", foreground="#555")
        self.hint.pack(anchor="w", pady=(4, 8))

        sec = ttk.Frame(self)
        sec.pack(fill="x")
        ttk.Label(sec, text="Secret partagé (HMAC)").pack(side="left")
        self.secret = ttk.Entry(sec, width=26, show="*")
        self.secret.insert(0, "usthb2026")
        self.secret.pack(side="left", padx=6)

        a = ttk.LabelFrame(self, text="Entité A", padding=6)
        a.pack(fill="both", expand=True, pady=6)
        self.plain = tk.Text(a, height=4, wrap="word")
        self.plain.pack(fill="both", expand=True)
        self.plain.insert("1.0", "Retrouve-moi a la bibliotheque a midi")
        ttk.Button(a, text="Chiffrer et envoyer", command=self.send).pack(anchor="e", pady=(4, 0))

        mid = ttk.LabelFrame(self, text="Canal (modifiable pour simuler un attaquant)", padding=6)
        mid.pack(fill="both", expand=True, pady=6)
        self.cipher = tk.Text(mid, height=3, wrap="word")
        self.cipher.pack(fill="both", expand=True)
        row = ttk.Frame(mid)
        row.pack(fill="x", pady=(4, 0))
        ttk.Label(row, text="Tag").pack(side="left")
        self.tag = ttk.Entry(row)
        self.tag.pack(side="left", fill="x", expand=True, padx=6)
        ttk.Button(row, text="Changer une lettre", command=self.tamper).pack(side="left")

        b = ttk.LabelFrame(self, text="Entité B", padding=6)
```

Partie 1 - Code de l'interface (app.py)

```
        b.pack(fill="both", expand=True, pady=6)
        self.result = tk.Text(b, height=4, wrap="word", state="disabled")
        self.result.pack(fill="both", expand=True)
        bar = ttk.Frame(b)
        bar.pack(fill="x", pady=(4, 0))
        self.status = ttk.Label(bar, text="")
        self.status.pack(side="left")
        ttk.Button(bar, text="Vérifier et déchiffrer", command=self.receive).pack(side="right")
        self.refresh_hint()

    def refresh_hint(self, _event=None):
        name = self.algo.get()
        self.key.delete(0, "end")
        if name == "Caesar":
            self.key.insert(0, "3")
            self.hint.config(text="Clé : décalage n (entier). Le sens s'applique au chiffrement.")
            self.direction.config(state="readonly")
        elif name == "Hill":
            self.key.insert(0, "3,3,2,5")
            self.hint.config(text="Clé : 4 ou 9 nombres (2x2 ou 3x3, ligne par ligne) ou un mot de
                4 / 9 lettres.")
            self.direction.config(state="disabled")
        else:
            self.key.insert(0, "monarchy")
            self.hint.config(text="Clé : un mot quelconque, J est confondu avec I.")
            self.direction.config(state="disabled")

    def encrypt(self, text):
        name = self.algo.get()
        k = self.key.get()
        if name == "Caesar":
            return ciphers.caesar(text, int(k), self.direction.get())
        if name == "Hill":
            return ciphers.hill_encrypt(text, parse_hill_key(k))
        return ciphers.playfair_encrypt(text, k)

    def decrypt(self, text):
        name = self.algo.get()
        k = self.key.get()
        if name == "Caesar":
            return ciphers.caesar_decrypt(text, int(k), self.direction.get())
        if name == "Hill":
            return ciphers.hill_decrypt(text, parse_hill_key(k))
        return ciphers.playfair_decrypt(text, k)

    def send(self):
        try:
            enc = self.encrypt(self.plain.get("1.0", "end").strip())
        except Exception as e:
            messagebox.showerror("Erreur", str(e))
            return
        self.cipher.delete("1.0", "end")
        self.cipher.insert("1.0", enc)
        self.tag.delete(0, "end")
        self.tag.insert(0, integrity.tag(enc, self.secret.get()))
        self.status.config(text="message envoyé", foreground="#1a6")

    def tamper(self):
        text = self.cipher.get("1.0", "end").strip()
        if not text:
            return
        i = len(text) // 2
        c = text[i]
        if c.isalpha():
            c = "A" if c.upper() != "A" else "B"
            text = text[:i] + c + text[i + 1:]
        self.cipher.delete("1.0", "end")
        self.cipher.insert("1.0", text)

    def receive(self):
        enc = self.cipher.get("1.0", "end").strip()
```

Partie 1 - Code de l'interface (app.py)

```
        if not integrity.verify(enc, self.secret.get(), self.tag.get().strip()):
            self.status.config(text="intégrité non vérifiée, message rejeté", foreground="#c22")
            self.set_result("")
            return
        try:
            dec = self.decrypt(enc)
        except Exception as e:
            messagebox.showerror("Erreur", str(e))
            return
        if self.algo.get() == "Playfair":
            dec = ciphers.playfair_strip_padding(dec)
        self.status.config(text="message authentique", foreground="#1a6")
        self.set_result(dec)

    def set_result(self, text):
        self.result.config(state="normal")
        self.result.delete("1.0", "end")
        self.result.insert("1.0", text)
        self.result.config(state="disabled")


class CipherAttackTab(ttk.Frame):
    def __init__(self, parent):
        super().__init__(parent, padding=10)
        ttk.Label(self, text="Texte chiffré").pack(anchor="w")
        self.box = tk.Text(self, height=5, wrap="word")
        self.box.pack(fill="x")
        self.box.insert("1.0", ciphers.caesar("Il faut garder le secret entre nous trois", 11))
        row = ttk.Frame(self)
        row.pack(fill="x", pady=6)
        ttk.Button(row, text="César : les 26 décalages", command=self.caesar_all).pack(side="left")
        ttk.Button(row, text="César : analyse de fréquence",
            command=self.caesar_freq).pack(side="left", padx=6)
        ttk.Button(row, text="Playfair : comptage des digrammes",
            command=self.digrams).pack(side="left")
        kp = ttk.LabelFrame(self, text="Hill 2x2 : attaque à clair connu", padding=6)
        kp.pack(fill="x", pady=6)
        ttk.Label(kp, text="Clair connu").grid(row=0, column=0, sticky="w")
        self.kp_plain = ttk.Entry(kp, width=30)
        self.kp_plain.insert(0, "MEETMEATNOON")
        self.kp_plain.grid(row=0, column=1, padx=6)
        ttk.Label(kp, text="Chiffré correspondant").grid(row=1, column=0, sticky="w")
        self.kp_cipher = ttk.Entry(kp, width=30)
        self.kp_cipher.insert(0, ciphers.hill_encrypt("MEETMEATNOON", [[3, 3], [2, 5]]))
        self.kp_cipher.grid(row=1, column=1, padx=6)
        ttk.Button(kp, text="Retrouver la clé", command=self.hill_attack).grid(row=0, column=2,
            rowspan=2, padx=6)
        self.out = tk.Text(self, height=14, wrap="word", state="disabled")
        self.out.pack(fill="both", expand=True)

    def show(self, text):
        self.out.config(state="normal")
        self.out.delete("1.0", "end")
        self.out.insert("1.0", text)
        self.out.config(state="disabled")

    def caesar_all(self):
        txt = self.box.get("1.0", "end").strip()
        self.show("\n".join("%2d  %s" % (k, t) for k, t in ciphers.caesar_bruteforce(txt)))

    def caesar_freq(self):
        txt = self.box.get("1.0", "end").strip()
        k = ciphers.caesar_frequency_guess(txt)
        self.show("La lettre la plus fréquente suggère le décalage %d\n\n%s" % (k,
            ciphers.caesar(txt, k, "gauche")))

    def digrams(self):
        txt = self.box.get("1.0", "end").strip()
        counts = ciphers.digram_counts(txt)[:15]
        self.show("\n".join("%s  %d" % (d, c) for d, c in counts) or "texte insuffisant")

```

Partie 1 - Code de l'interface (app.py)

```
    def hill_attack(self):
        try:
            k = ciphers.hill_known_plaintext(self.kp_plain.get(), self.kp_cipher.get(), 2)
        except ValueError as e:
            self.show(str(e))
            return
        self.show("Matrice clé retrouvée :\n%s\n%s" % (k[0], k[1]))


SCENARIOS = {
    "3 caractères, chacun 0 ou 1": (passwords.BINARY, 3),
    "4 caractères, chacun de 1 à 4": (passwords.DIGITS_1_4, 4),
    "caractères a-z A-Z 0-9 + * ...": (passwords.FULL, 3),
}


class PasswordTab(ttk.Frame):
    def __init__(self, parent):
        super().__init__(parent, padding=10)
        top = ttk.Frame(self)
        top.pack(fill="x")
        ttk.Label(top, text="Cas").grid(row=0, column=0, sticky="w")
        self.case = ttk.Combobox(top, values=list(SCENARIOS), state="readonly", width=32)
        self.case.current(0)
        self.case.grid(row=0, column=1, padx=6)
        self.case.bind("<<ComboboxSelected>>", self.on_case)
        ttk.Label(top, text="Longueur").grid(row=0, column=2, padx=(12, 0))
        self.length = tk.Spinbox(top, from_=1, to=4, width=4)
        self.length.grid(row=0, column=3, padx=6)
        ttk.Label(top, text="Mot de passe cible").grid(row=1, column=0, sticky="w", pady=6)
        self.target = ttk.Entry(top, width=24)
        self.target.insert(0, "101")
        self.target.grid(row=1, column=1, sticky="w", padx=6)
        row = ttk.Frame(self)
        row.pack(fill="x", pady=6)
        ttk.Button(row, text="Attaque par dictionnaire", command=lambda:
            self.go("dictionary")).pack(side="left")
        ttk.Button(row, text="Force brute", command=lambda: self.go("brute")).pack(side="left",
            padx=6)
        ttk.Button(row, text="Dictionnaire puis force brute", command=lambda:
            self.go("both")).pack(side="left")
        self.out = tk.Text(self, height=18, wrap="word", state="disabled")
        self.out.pack(fill="both", expand=True)
        self.busy = False
        self.on_case()

    def on_case(self, _event=None):
        cs, ln = SCENARIOS[self.case.get()]
        self.length.delete(0, "end")
        self.length.insert(0, str(ln))
        self.target.delete(0, "end")
        self.target.insert(0, cs[1] * ln)

    def write(self, text):
        self.out.config(state="normal")
        self.out.insert("end", text + "\n")
        self.out.see("end")
        self.out.config(state="disabled")

    def go(self, mode):
        if self.busy:
            return
        cs, _ = SCENARIOS[self.case.get()]
        pw = self.target.get()
        try:
            ln = int(self.length.get())
        except ValueError:
            return
        if len(pw) != ln or not set(pw) <= set(cs):
            messagebox.showwarning("Mot de passe", "Le mot de passe doit avoir %d caractères pris
                dans l'ensemble autorisé." % ln)
```

Partie 1 - Code de l'interface (app.py)

```
            return
        space = passwords.search_space(cs, ln)
        self.write("empreinte cible %s...  espace de recherche %d" % (passwords.sha(pw)[:16],
            space))
        self.busy = True
        threading.Thread(target=self.work, args=(mode, pw, cs, ln), daemon=True).start()

    def work(self, mode, pw, cs, ln):
        h = passwords.sha(pw)
        if mode == "dictionary":
            found, tries, secs = passwords.dictionary_attack(h)
            how = "dictionnaire"
        elif mode == "brute":
            found, tries, secs = passwords.brute_force(h, cs, ln)
            how = "force brute"
        else:
            found, tries, secs, how = passwords.combined_attack(h, cs, ln)
        if found is None:
            msg = "%s : introuvable après %d essais (%s)" % (how, tries, passwords.human_time(secs))
        else:
            msg = "%s : '%s' trouvé après %d essais (%s)" % (how, found, tries,
                passwords.human_time(secs))
        self.after(0, self.finish, msg)

    def finish(self, msg):
        self.busy = False
        self.write(msg)


class ProtectionTab(ttk.Frame):
    def __init__(self, parent):
        super().__init__(parent, padding=10)
        row = ttk.Frame(self)
        row.pack(fill="x")
        ttk.Label(row, text="Mot de passe").pack(side="left")
        self.pw = ttk.Entry(row, width=26)
        self.pw.insert(0, "Tr0ub4dor*93x")
        self.pw.pack(side="left", padx=6)
        ttk.Button(row, text="Tester la robustesse", command=self.check).pack(side="left")
        ttk.Button(row, text="Mesurer le hachage lent", command=self.timing).pack(side="left",
            padx=6)
        ttk.Button(row, text="Démo du verrouillage", command=self.lockout).pack(side="left")
        ttk.Button(row, text="Tableau des temps d'attaque", command=self.table).pack(side="left",
            padx=6)
        self.out = tk.Text(self, height=22, wrap="word", state="disabled")
        self.out.pack(fill="both", expand=True, pady=8)

    def show(self, text):
        self.out.config(state="normal")
        self.out.delete("1.0", "end")
        self.out.insert("1.0", text)
        self.out.config(state="disabled")

    def check(self):
        p = passwords.strength_problems(self.pw.get())
        self.show("Accepté." if not p else "Refusé :\n- " + "\n- ".join(p))

    def timing(self):
        import time
        rec = passwords.make_record(self.pw.get())
        t = time.perf_counter()
        passwords.check_record(rec, "wrong")
        one = time.perf_counter() - t
        fast = passwords.hash_rate(50000)
        self.show("Une vérification PBKDF2 (%d tours) : %s\nSHA-256 simple : environ %d essais par
            seconde\n"
                  "Hachage lent : environ %d essais par seconde\nFacteur de ralentissement :
                      environ %dx" %
                  (rec["iterations"], passwords.human_time(one), fast, 1 / one, fast * one))

    def lockout(self):
```

Partie 1 - Code de l'interface (app.py)

```
        rec = passwords.make_record("correct", 2000)
        g = passwords.LoginGuard(rec, 3, 30)
        lines = []
        t = 0.0
        for a in ["aaa", "bbb", "ccc", "correct", "correct"]:
            if a == "correct" and len(lines) == 4:
                t += 31
            lines.append("t=%5.1fs  essai '%s'  ->  %s" % (t, a, g.try_login(a, now=t)))
            t += 1
        self.show("\n".join(lines))

    def table(self):
        rate = passwords.hash_rate(50000)
        lines = ["Temps moyen pour trouver le mot de passe, SHA-256 simple à %d essais/s\n" % rate]
        for label, (cs, ln) in SCENARIOS.items():
            sp = passwords.search_space(cs, ln)
            lines.append("%-34s %12d candidats  %s" % (label, sp,
                passwords.human_time(passwords.estimate_seconds(sp, rate))))
        full = len(passwords.FULL)
        for ln in (6, 8, 10, 12):
            sp = full ** ln
            lines.append("%-34s %12.2e candidats  %s" % ("ensemble complet, %d car." % ln, sp,
                passwords.human_time(passwords.estimate_seconds(sp, rate))))
        self.show("\n".join(lines))


class Window(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Framework de chiffrement - SSAD TP1")
        self.geometry("820x720")
        nb = ttk.Notebook(self)
        nb.pack(fill="both", expand=True, padx=6, pady=6)
        nb.add(MessagingTab(nb), text="Messagerie")
        nb.add(CipherAttackTab(nb), text="Attaques sur les chiffrements")
        nb.add(PasswordTab(nb), text="Attaques sur mots de passe")
        nb.add(ProtectionTab(nb), text="Protection")


if __name__ == "__main__":
    Window().mainloop()

```